

Supplementary Material 1 for: Continuous curation of an *Alteromonas macleodii* metabolic model

Helen Scott, Devlin Moyer, Osnat Weissberg, Franziska Kratzl, Waseem Bashir
Valiya Kalladi, Rogier Braakman, Mary Ann Moran, Daniel Sher, Daniel Segrè

Contents

1	Motivation	2
1.1	Why do we need this?	2
1.2	What have people been doing before this?	2
1.2.1	MEMOTE	2
1.2.2	standard-GEM	2
1.2.3	human-GEM and yeast-GEM	2
1.3	What's new here	2
2	Set-Up	3
2.1	What is GitHub?	3
2.1.1	Diffs	3
2.2	What's in a Name?: Choosing your Model and Repository Name	5
2.3	Branches	5
2.4	Repository Structure	6
2.4.1	Model	6
2.4.2	Data	6
2.5	GitHub Actions	6
2.5.1	Defining an Action with a YML file	6
3	The Continuous Curation Loop	6
3.1	Step 1) Curate	6
3.1.1	Removing Reactions/Metabolites	7
3.1.2	Pull Requests	7
3.2	Step 2) Test	7
3.2.1	What is a Unit Test?	7
3.2.2	What makes a Good Unit Test?	8
3.2.3	Examples of Unit Tests for Tool Testing	9
3.2.4	Examples of Unit Tests for Model Curation	9
3.2.5	Examples of Unit Tests for Enforcement	10
3.3	Step 3) Report	11
3.3.1	Scripts vs Tests	11
3.3.2	Examples of Scripts Used for Model Curation	11
3.4	Step 4) Release	11
3.5	Step 5) Run	12
3.6	Step 6) Monitor	12
4	Why can't you just make it for me?	12
5	What's the cost?	12

6	How to think like a Programmer	12
6.1	Questions to Ask Yourself:	12
7	Glossary	12
8	References	14

GEMs are, at their heart, a software product, and we took lessons from software development and applied them to the model curation process. We term this constant testing and iterative model improvement strategy “continuous curation”, inspired by continuous integration (CI) for traditional software. This included tracking all changes using version control, having multiple curators collaborate and propose changes by working on branches and opening pull requests, testing the model with defined pass/fail tests, and automatically generating artifacts for curator inspection.

- None of this is really new
 - we borrowed all of this from software engineering
 - other groups do or are developing similar things
 - we took some things directly from human-GEM and standard-GEM
 - the field is crystallizing
 - but the field lacks a single reference point for guidelines

1 Motivation

1.1 Why do we need this?

- GEMs are important tools
 - But their quality is often questioned
- Manual curation is hard/messy
 - Manual curation can be an overwhelming task- the typical advice of just go “pathway by pathway” can be paralyzingly large
- Manual curation can last for a long time (including decades, spanning many people and projects)
 - Often one group publishes a model, then another may find it, modify it, and publish a new paper, resulting in branching sets of models

1.2 What have people been doing before this?

1.2.1 MEMOTE

- MEMOTE exists
 - but it’s more of a benchmarking tool- a lot of things is just about the file
 - test what the custom tests ever did

1.2.2 standard-GEM

- no tests

1.2.3 human-GEM and yeast-GEM

- some tests
- A lot of things were not enforced, just reminders and check boxes

1.3 What’s new here

- Our specific unit tests- not really clear who go to running it as CI first. . .
- Biomass component producibility heatmaps
- Improved tracking removed reactions/metabolites

2 Set-Up

2.1 What is GitHub?

Version control is critical for model curation because it tracks the “who”, “what”, and “when” of all changes made to the model. Who edited the model file, when did they make the edits, and what exactly was changed. It also maintains the historical versions of the model file, so at any time you can revert changes and return to an older version of the model.

Version control keeps a historical record of changes made to tracked files in a specialized database called a repository (or “repo”). Git is the software tool that enables version control, and GitHub is one popular cloud-based platform to host Git repositories, that also offers other functionalities such as issue tracking and wiki hosting. While we used GitHub, and will use GitHub-focused terminology (e.g., pull requests, actions) it is not the only option for hosting Git repositories, other popular options include GitLab, Bitbucket and Azure DevOps, each of which have analogous tools to those we describe here and could similarly be used for a continuous curation pipeline.

- A Git repository lives in two places...
- Local vs Remote, basic terminology (commit, push, pull, etc.)

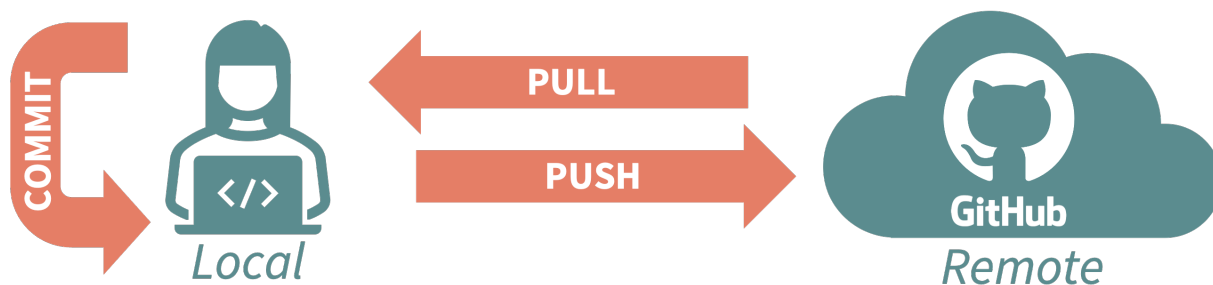


Figure 1: An introduction to basic Git terminology: Local and remote repositories, commit, push, and pull

2.1.1 Diffs

- How to read a diff

Commit e8c0690 Preview Give feedback Browse files

hgscott committed 6 hours ago

Remove more nan formulae

main 1 parent ff92fbf commit e8c0690

Filter files... 1 file changed +2 -2 lines changed Search within code

@@ -7399,7 +7399,7 @@			
7399			</html>
7400			</notes>
7401			</species>
7402	-		<species id="M_cpd27484_c0" name="MPT-Synthases [c0]" compartment="c0" hasOnlySubstanceUnits="false" boundaryCondition="false" constant="false" fbc:charge="0" fbc:chemicalFormula="nan">
7403			<notes>
7404			<html
7405			xmlns="http://www.w3.org/1999/xhtml">
			<p>modelseed_template_id:
			cpd27484_c</p>

- Not all diffs are equally helpful
 - File types to avoid
 - Microsoft office files: e.g., .xlsx

Excel

Commit 5fd631e

Preview

Give feedback

 Browse files

 hgscott committed on Jan 24 ·  0 / 1

Expand the annotation cols

 main

1 parent [ce4f625](#) commit 5fd631e 



 **2 files changed** +16 -0 lines changed

> export_model.py  

+16 ■■■■■ ...

▼ model.xlsx 

40.2 KB ...

Binary file not shown.

CSV

[illegible]

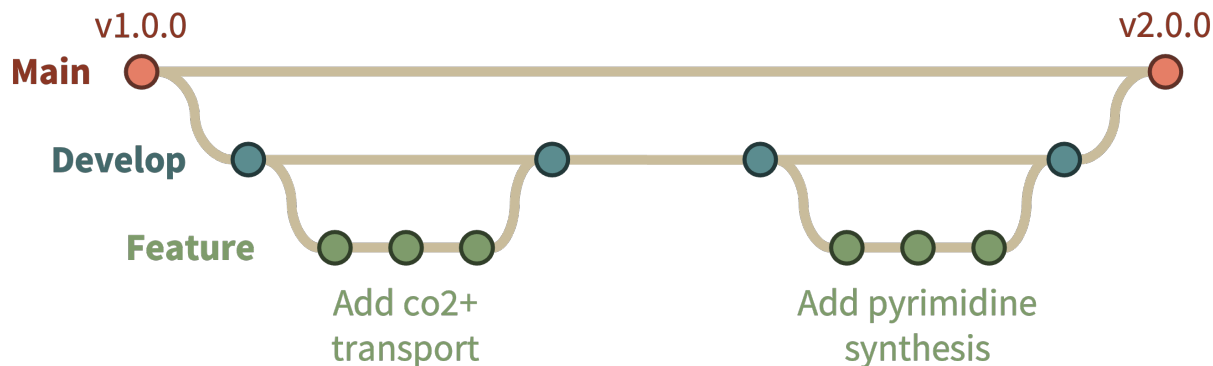
For a more in depth coverage, see...

2.2 What's in a Name?: Choosing your Model and Repository Name

- What does BiGG/Palsson do
- What does standard-GEM do

2.3 Branches

Branching is a key feature of Git- it allows developers to isolate their changes so that the main version of the repository is not affected. This allows multiple developers to work simultaneously, and allows developers to test out changes where they will not affect anyone else. We chose to use a branching strategy based on the popular GitFlow strategy. We had two long-lived branches, “main”, the main branch, which had the official releases of the model, and “dev”, the development branch, where all accepted changes to the model were integrate before an official release. All changes made the model were made on feature branches that branched off of and were merged back into the dev branch. This ensured that any new feature development did not disturb the main model. Early on in development, branching is critical to XXX, and later branching became increasingly important to differentiate the version of model from users vs from developers.



2.4 Repository Structure

2.4.1 Model

2.4.1.1 What file type to use?

2.4.2 Data

2.5 GitHub Actions

We used automation through GitHub actions to run tests and scripts upon the opening of a pull request.

2.5.1 Defining an Action with a YML file

3 The Continuous Curation Loop

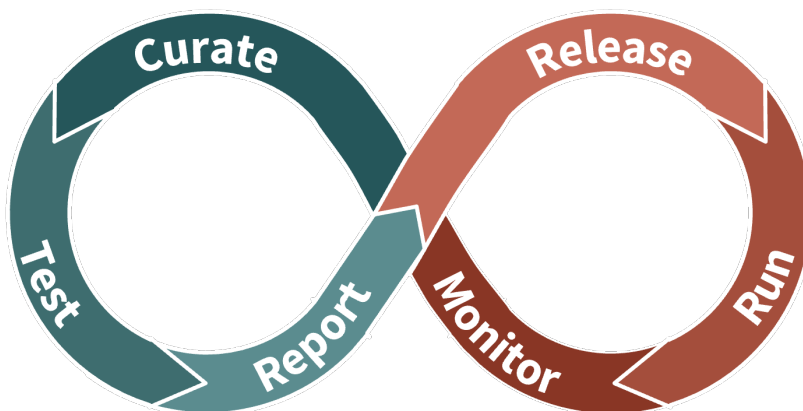


Figure 2: Continuous Curation is an iterative process with the following steps: Curate, Test, Report, Release, Run, and Monitor.

The Continuous Curation loop consists of 6 steps: 1. Curate 2. Test 3. Report 4. Release 5. Run 6. Monitor

3.1 Step 1) Curate

NOTE: We do not discuss here how to make curation decisions, but rather how to implement them * How big is one curation task?

3.1.1 Removing Reactions/Metabolites

- Palsson said to do it
- What we took from human-GEM: the table
- What is new
 - The standard vocabulary of reason
 - The list of removed things in the model file itself
 - And the test to make sure it does not drift
 - The test that no old reactions are still in the model file

3.1.2 Pull Requests

One critical component of the history of changes to the model is the “why”- why was a change to the model made (e.g., was a reaction found to have genomic evidence, was there a mistake in the biochemistry database, etc.). There are text fields in the model file itself where this information can be stored, and there have been cases in the past of defined “codes” used to represent different types of evidence that support each reaction (CITE EXAMPLES) however we have found that these are not well used, lack standardization across the community, and are often not comprehensive enough to fully explain the reasoning behind each change. We instead elected to document these in issues and pull requests on the repository. Issues can be used as a sort of electronic lab notebook. To ensure that all curators (present and future) are reminded to document their reasoning, a pull request template was used.

- Open a pull request, that starts the cycle

3.2 Step 2) Test

- Testing code is important, testing the model is just as important
- Typical software tools can be used, but some concepts need to be generalized
- Unit tests are considered critical to the success of any project

3.2.1 What is a Unit Test?

- Unit tests are a common software development practice in which the smallest individual parts of the code (called units) are individually tested, to ensure each gives the expected outcome

3.2.1.1 Software example

- Imagine you have a python module called `hello` with a single function, also called `hello`, that says “Hello” to a person, given their name:

```
def hello(name: str):  
    message = 'Hello, ' + name + '!'   
  
    return message
```

- To write the unit test, you would make a new file, by convention in a directory named “test” or “tests”, and name the file “test_[module name]”, for that individual python module, so in our case “test_hello”
- While there are other testing frameworks, we show the python standard-library testing framework, `unittest`
- In your test, you first import `unittest`, and your function that you are testing

```
import unittest
```

```
from hello import hello
```

- All tests are put within a class

```
class TestHello(unittest.TestCase):
```

- Within that class you can write multiple tests
- For example, first test an expected input, e.g., “Helen”
 - ```
def test_helen(self):
 # Define a name
 name = 'Helen'
 # Call the function
 message = hello(name)

 # Assert that the function returns the correct string
 self.assertEqual(message, 'Hello, Helen!')
```
  - You define if the output is correct or not with an assert statement
  - There are other options for assert statements that make sense for different kinds of tests
    - Table/list of assert statements
- Then test an edge case, something that could return a bad output if your function is not properly written, e.g. a number

*# Add a test checking that the function fails when an integer is passed*

```
def test_integer(self):
 # Define a name
 name = 123
 # Call the function
 with self.assertRaises(TypeError):
 message = hello.hello(name)
```

- At the end of the file- set it to run all of the classes

```
if __name__ == '__main__':
 unittest.main()
```

- Unit tests can be run manually- e.g., a developer runs them on their own computer and verifies that they all pass before pushing code
- Or they can be run automatically (e.g., as part of a GitHub action)
- In GEM-MIT1002 we run them all automatically in the CI-workflow, finding and executing tests with pytest

### 3.2.2 What makes a Good Unit Test?

- Needs to pass/fail, have an expected outcome, not generate an artifact
  - Boolean result- pass or fail
  - Self-validating — it returns pass or fail, not output a human must inspect
- One reason to fail — a test asserting five things tells you “something broke,” not what
- Fast — slow tests don’t get run, and a suite people skip provides no safety
- Independent — no test depends on another’s state or on run order
- Repeatable — same result on any machine, any time; no network, no clock, no randomness
- Deterministic — the intermittent test is worse than no test, because it trains people to re-run until green
- No logic in the test — conditionals and loops inside a test can themselves be buggy, and then you’re debugging your test
- Tests behavior, not implementation — a test that breaks when you refactor without changing behavior is a liability
- A name that says what broke without opening the file

- Table with columns:
  - Example of good unit tests
    - Test that a script generate the correct data and saves a plot with the correct path
  - Example of bad unit tests
    - Generate a figure, that you need to look at
    - A test you know will fail, and you will just ignore it (skip the test or mark it a known failure instead)

### 3.2.3 Examples of Unit Tests for Tool Testing

Part of the GEM-MIT1002 repo is helper functions (i.e. for XXX), these functions, just like any other functions in a python module should be tested. For example: \* ...

### 3.2.4 Examples of Unit Tests for Model Curation

- In traditional software engineering, the unit being tested is often a function, however for the case of model curation, we are testing the model as a whole, but can write tests to focus on individual aspects of the model
- The ones we present here are by no means an exhaustive list of everything that could or should be tested.
- Many of these tests use previously published tools (e.g. MEMOTE), but we found that by implementing them with unit tests on a GitHub action it was easier to track model performance over time and recognize errors introduced into the model quickly.
- `test_biomass.py`
  - `TestBiomass`
    - `test_biomass_weight:`
      - Tests that the biomass metabolite (`cpd11416_c0`) added up to 1 g, this is important for dFBA simulations.
- `test_cycles.py:`
  - `TestCycles`
    - `test_atp_generating_cycles`
      - Tests that there are no erroneous energy generating cycles capable of regenerating ATP without an input carbon source, using the MEMOTE function `consistency.detect_energy_generating_cycles()`
- `test_exchanges.py:`
  - `TestExchanges`
    - `test_dead_end_transporters`
      - Checks that there are no dead-end transporters (i.e., external metabolites without an exchange reaction).
- `test_growth.py:`
  - `TestGrowthWithoutCarbon`
    - `test_no_growth_without_carbon`
      - We checked that the model was not capable of growth without a carbon source in the medium.
  - `TestExpectedGrowthPhenotypes`
    - `test_no_new_mismatches`
      - And that the model recapitulated all known experimental growth phenotypes
        - depending on exactly how you implement this, it might “fail” for the majority of time of curation.
        - We made sure that the model never got worse by...
    - `test_no_stale_baseline_entries`
    - `test_mismatch_categories_are_unchanged`
    - `test_excluded_rows_are_not_scored`
    - `test_every_phenotype_was_evaluated`

- TestSummaryArithmetic
  - test\_every\_row\_gets\_a\_known\_category
  - test\_categories\_partition\_the\_table
  - test\_no\_uptake\_route\_is\_a\_subset\_of\_the\_negative\_predictions
  - test\_a\_missing\_exchange\_does\_not\_by\_itself\_decide\_the\_verdict
  - test\_confusion\_matrix\_sums\_to\_the\_scored\_rows
  - test\_matches\_are\_the\_concordant\_cells
  - test\_matches\_never\_exceed\_the\_interpretable\_denominator
  - test\_excluded\_rows\_keep\_their\_underlying\_verdict
- test\_sbml.py:
  - TestValidSBML
    - test\_valid\_sbml
      - We tested that the SBML file was valid- important as COBRAPy may fail to load a model with a malformed file, and KBase created such files.
    - test\_isolated\_genes\_and\_mets
      - We tested for isolated genes and metabolites.
    - test\_mass\_balance
      - We tested that all reactions were mass and charge balanced.

### 3.2.5 Examples of Unit Tests for Enforcement

- test\_deprecated.py
  - TestDeprecatedNotInModel
    - test\_deprecated\_reactions\_absent
    - test\_deprecated\_metabolites\_absent
  - TestDeprecatedSchema
    - test\_headers\_exact
    - test\_no\_duplicate\_ids
    - test\_reasons\_in\_vocabulary
    - test\_ids\_have\_no\_sbml\_prefix
    - test\_dates\_are\_iso
    - test\_pr\_format
    - test\_notes\_are\_single\_line
  - TestReplacedBy
    - test\_separator\_is\_semicolon
    - test\_targets\_look\_like\_identifiers
    - test\_no\_sbml\_prefix\_on\_targets
    - test\_no\_duplicate\_targets
    - test\_required\_when\_reason\_is\_relative
    - test\_not\_self\_referential
    - test\_targets\_resolve
  - TestNotesMirror
    - test\_mirror\_matches\_tsv
    - test\_pointer\_present
    - test\_mirror\_survives\_cobrapy\_round\_trip
  - TestStampPrNumber
    - test\_fills\_blanks\_and\_preserves\_existing
    - test\_is\_idempotent
    - test\_rejects\_non\_numeric
  - TestIdentifierListParsing
    - test\_accepts\_common\_separators
    - test\_strips\_sbml\_prefixes
    - test\_empty\_is\_empty
    - test\_normalizes\_to\_semicolons

- test\_record\_normalizes\_on\_construction
  - test\_preserves\_gene\_style\_identifiers
- TestVocabularyDocumented
  - test\_every\_reason\_in\_readme
- test\_phenotype\_data
  - TestPhenotypeSchema
    - test\_growth\_column\_vocabulary
    - test\_exclude\_reason\_vocabulary
    - test\_every\_reason\_is\_documented
    - test\_condition\_keys\_are\_unique
    - test\_met\_ids\_are\_present\_and\_well\_formed
  - TestInterpretableCount
    - test\_categories\_are\_distinct
    - test\_interpretable\_excludes\_unsure\_and\_excluded\_rows
    - test\_interpretable\_is\_a\_strict\_subset\_when\_rows\_are\_held\_out
  - TestExpectedMisMatches
    - test\_baseline\_has\_the\_expected\_columns
    - test\_baseline\_rows\_refer\_to\_real\_conditions
    - test\_baseline\_does\_not\_list\_excluded\_conditions

### 3.3 Step 3) Report

#### 3.3.1 Scripts vs Tests

#### 3.3.2 Examples of Scripts Used for Model Curation

- Also ran through GitHub actions, were a set of “scripts”. These differ from tests because they cannot “pass” or “fail”, and instead generate artifacts (e.g., plots) for a human curator to look at.
- Using the same underlying code as the growth test, we generated a plot of which experimentally known growth phenotypes the model matched or not. The heatmap visualization was useful for sharing results.
- While this graph is useful to grasp model performance at a glance, it was not the most instructive when gap-filling the model (just knowing that the model does not grow does not help you find a gap). Instead we checked the model’s ability to produce each individual biomass component (i.e., we added a demand/sink reaction for each biomass component that take the metabolite and removes it from the system (similar to an exchange reaction), and looped through the list of biomass components and set each as the objective, maximizing the flux through that sink reaction. A positive value indicated that the model was capable of producing that biomass component. This helped narrow down searches for gaps (e.g., could say that a subset of amino acids was not producible, therefore there must be a gap in that pathway). When using an objective other than the biomass, we considered if there should be free transport/exchange/sinks for all biomass components simultaneously or if only the one being maximized should have a sink. Theoretically, there could be components whose production is tied and without flux through the biomass reaction, dead ends could appear that block flux.

### 3.4 Step 4) Release

- What counts as a new version
- Chores upon release
  - different file types
  - MEMOTE
  - MACAW
- GitHub release
- Zenodo release

### 3.5 Step 5) Run

- This is the fun part, where you, or others, actually try to use the model

### 3.6 Step 6) Monitor

- Open an issue
- What belongs in an issue

## 4 Why can't you just make it for me?

- Can't you just make an installable tool that makes all of this for me?

## 5 What's the cost?

## 6 How to think like a Programmer

### 6.1 Questions to Ask Yourself:

1.

## 7 Glossary

- **Artifact:**
  - *In software engineering:*
  - *In continuous curation:*
- **Branch:** A branch is a parallel version of a repository. It is contained within the repository, but does not affect the primary or main branch allowing you to work freely without disrupting the “live” version. <sup>1</sup>.
- **Commit:** A commit, or “revision”, is an individual change to a file (or set of files). When you make a commit to save your work, Git creates a unique ID (a.k.a. the “SHA” or “hash”) that allows you to keep record of the specific changes committed along with who made them and when. Commits usually contain a commit message which is a brief description of what changes were made. <sup>2</sup>.
- **Commit Message:** Short, descriptive text that accompanies a commit and communicates the change the commit is introducing <sup>3</sup>.
- **Conflict:** A situation where two branches have changes in a file that Git cannot automatically merge, requiring manual resolution <sup>4</sup>.
- **Continuous Integration (CI):** A development practice where team members frequently integrate their code into a shared repository, often multiple times a day. Each integration is verified by automated builds and tests to detect errors early <sup>5</sup>.
- **Continuous Delivery:** A software development practice where teams keep their product in a deployable state at all times, but deployment still requires a manual decision <sup>6</sup>.
- **Continuous Deployment:** A software development practice where every change that passes all automated tests is released to production automatically <sup>7</sup>.
- **Curation:**
  - *In continuous curation:*

---

<sup>1</sup><https://docs.github.com/en/get-started/learning-about-github/github-glossary>

<sup>2</sup><https://docs.github.com/en/get-started/learning-about-github/github-glossary>

<sup>3</sup><https://docs.github.com/en/get-started/learning-about-github/github-glossary>

<sup>4</sup><https://informatics.fas.harvard.edu/resources/glossary/#git-terms>

<sup>5</sup><https://www.agile-academy.com/en/agile-dictionary/>

<sup>6</sup><https://www.agile-academy.com/en/agile-dictionary/>

<sup>7</sup><https://www.agile-academy.com/en/agile-dictionary/>

- **Diff:** A diff is the difference in changes between two commits, or saved changes. The diff will visually describe what was added or removed from a file since its last commit <sup>8</sup>.
- **Feature Branch:** A branch used to experiment with a new feature or fix an issue that is not in production. Also called a topic branch <sup>9</sup>.
- **Git:** Git is an open source program for tracking changes in text files <sup>10</sup>.
- **GitFlow:**
- **GitHub:** A web-based platform that facilitates Git's use for collaboration between individuals. Other web-based platforms include GitLab and BitBucket <sup>11</sup>.
- **GitHub Actions:**
- **Issue:** Issues are suggested improvements, tasks or questions related to the repository. Issues can be created by anyone (for public repositories), and are moderated by repository collaborators. Each issue contains its own discussion thread. You can also categorize an issue with labels and assign it to someone <sup>12</sup>.
- **JSON:**
- **Local:**
- **Merge:** Merging takes the changes from one branch (in the same repository or from a fork), and applies them into another. This often happens as a "pull request" (which can be thought of as a request to merge), or via the command line <sup>13</sup>.
- **Monitor:**
  - *In software engineering:*
  - *In continuous curation:*
- **Pull:** The process of integrating changes from one version of a repository to another (e.g. from a fork back to the original repo, or from a branch back to the main branch). There are two general use cases: 1) When the owner of a repository makes changes to it, you pull those changes into your local copy. 2) When you make changes to a forked repository or a branch of a repository and want to incorporate the changes back to the original repo or branch, you initiate a pull request, and then whoever is in charge of the original repository can pull those changes in <sup>14</sup>.
- **Pull Request:** Pull requests are proposed changes to a repository submitted by a user and accepted or rejected by a repository's collaborators <sup>15</sup>.
- **Push:** To push means to send your committed changes to a remote repository on GitHub.com. For instance, if you change something locally, you can push those changes so that others may access them <sup>16</sup>.
- **Release:**
  - *In software engineering:*
  - *In continuous curation:*
- **Remote Repository:** This is the version of a repository or branch that is hosted on a server, most likely GitHub.com <sup>17</sup>.
- **Report:**
  - *In software engineering:*
  - *In continuous curation:*
- **Repository/Repo:** A repository is the most basic element of GitHub. They're easiest to imagine as a project's folder. A repository contains all of the project files (including documentation), and stores each file's revision history. Repositories can have multiple collaborators and can be either public or private <sup>18</sup>.
- **Run:**

<sup>8</sup><https://docs.github.com/en/get-started/learning-about-github/github-glossary>

<sup>9</sup><https://docs.github.com/en/get-started/learning-about-github/github-glossary>

<sup>10</sup><https://docs.github.com/en/get-started/learning-about-github/github-glossary>

<sup>11</sup><https://informatics.fas.harvard.edu/resources/glossary/#git-terms>

<sup>12</sup><https://docs.github.com/en/get-started/learning-about-github/github-glossary>

<sup>13</sup><https://informatics.fas.harvard.edu/resources/glossary/#git-terms>

<sup>14</sup><https://informatics.fas.harvard.edu/resources/glossary/#git-terms>

<sup>15</sup><https://docs.github.com/en/get-started/learning-about-github/github-glossary>

<sup>16</sup><https://docs.github.com/en/get-started/learning-about-github/github-glossary>

<sup>17</sup><https://docs.github.com/en/get-started/learning-about-github/github-glossary>

<sup>18</sup><https://docs.github.com/en/get-started/learning-about-github/github-glossary>

- *In software engineering:*
  - *In continuous curation:*
- **SBML:**
- **Script:**
  - *In software engineering:*
  - *In continuous curation:*
- **Semantic Versioning:**
- **Test:**
  - *In software engineering:*
  - *In continuous curation:*
- **Trunk-Based Development:**
- **Version Control:** The process of tracking changes to files over time, allowing you to recall specific versions later <sup>19</sup>.
- **XML:**

## 8 References

---

<sup>19</sup><https://informatics.fas.harvard.edu/resources/glossary/#git-terms>